

Chapter 8 Binomial GLM

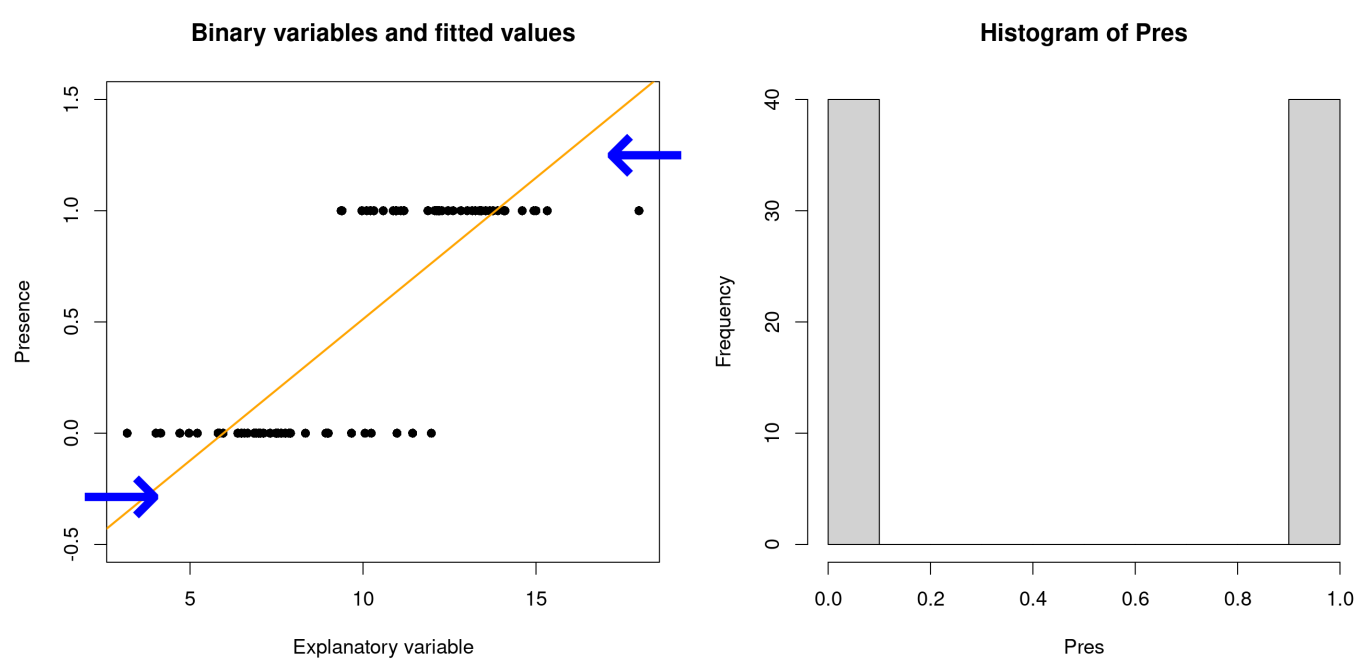
A common response variable in ecological data sets is the binary variable: we observe a phenomenon Y or its "absence". For example, species presence/absence is frequently recorded in ecological monitoring studies. We usually wish to determine whether a species' presence is affected by some environmental variables. Other examples include the presence/absence of a disease within a wild population, the success/failure to record a specific behaviour, and the survival/death of organisms. Usually, we are interested in questions such as: how do species occurrences vary in function of the environment?

$$Occurrences = f(Environment)$$

Under a linear model, expected values can be out of the $[0, 1]$ range with $\ln()$:

```
# set up some binary data
Pres <- c(rep(1, 40), rep(0, 40))
rnorm <- function(x) rnorm(1, mean = ifelse(x == 1, 12.5, 7.5), sd = 2)
ExpVar <- sapply(Pres, rnorm)

# Linear model with binary data...
lm(Pres ~ ExpVar)
```



When we predict the probability to observe a phenomenon Y with a binary variable, the predicted value has to be between 0 and 1: it is the possible range of probability!

8.1 GLM with binomial data: logit link

As we have seen in the previous section, a regression that has a binary response variable is one of many generalized linear models and is called a logistic regression or a logit model.

A **generalized linear model** is made of a **linear predictor**:

$$\underbrace{g(\mu_i)}_{\text{Link function}} = \underbrace{\beta_0 + \beta_1 X_{i1} + \dots + \beta_p X_{ip}}_{\text{Linear component}}$$

Consider that $Y_i \sim B(n_i, p_i)$, and that we want to model the proportions of Y_i/n_i . As such:

$$E(Y_i/n_i) = p_i \text{ and } \text{var}(Y_i/n_i) = \frac{1}{n_i} p_i(1 - p_i) \text{ , so that } V(\mu_i) = \mu_i(1 - \mu_i)]$$

We first move the probabilities μ_i to the odds:

$$\text{odds}_i = \frac{\mu_i}{1 - \mu_i}$$

The odds puts our expected values on a 0 to $+\text{Inf}$ scale.

We then take logarithms, calculating the *logit* or log-odds:

$$\eta_i = \text{logit}(\mu_i) = \log\left(\frac{\mu_i}{1 - \mu_i}\right)$$

with μ being the expected values (probability that $Y = 1$), and with the expected values now ranging from $-\text{Inf}$ to $+\text{Inf}$.

In R, presence (or success, survival...) is usually coded as 1 and absence (or failure, death...) as 0. A logistic regression (or any other generalized linear model) is performed with the `glm()` function. This function is different from the basic `lm()` as it allows one to specify a statistical distribution other than the normal distribution.

```
glm(formula,
     family = ???, # this argument allows us to set a probability distribution!
     data,
     ...)
```

We have seen that binary variables are not normally distributed (i.e. we observe a peak at 0 and a peak at 1, but nothing in between). In R, we can specify the statistical distribution of our model with the argument 'family'. For a logistic regression, we indicate 'family = binomial'.

Distribution of Y	Link function name	Link function	Model	R
Normal	Identity	$g(\mu) = \mu$	$\mu = \mathbf{X}\beta$	gaussian(link="identity")
Binomial	Logit	$g(\mu) = \log\left(\frac{\mu}{1-\mu}\right)$	$\log\left(\frac{\mu}{1-\mu}\right) = \mathbf{X}\beta$	binomial(link="logit")
Poisson	Log	$g(\mu) = \log(\mu)$	$-\mu^{-1} = \mathbf{X}\beta$	poisson(link="log")
Exponential	Negative Inverse	$g(\mu) = -\mu^{-1}$	$\log(\mu) = \mathbf{X}\beta$	gamma(link="inverse")

In R, we can therefore build a binomial GLM with a logit link as follows:

```
# This is the syntax for a binomial GLM with a logit link
glm(formula,
     family = binomial(link = "logit"), # this is also known as logistic
     data,
     ...)
```

8.2 Example

Let's build our first generalized linear model! Here, we want to build a logistic regression model using the `mites` data

```
# Exercise 1 - our first GLM!
# setup('...')

mites <- read.csv("data/mites.csv", header = TRUE)
str(mites)

## 'data.frame': 70 obs. of 9 variables:
## $ Galumna : int 8 3 1 1 2 1 1 1 2 5 ...
## $ pa : int 1 1 1 1 1 1 1 1 1 1 ...
## $ totalabund: int 140 268 186 286 199 209 162 126 123 166 ...
## $ prop : num 0.05714 0.01119 0.00538 0.0035 0.01005 ...
## $ SubsDens : num 39.2 55 46.1 48.2 23.6 ...
## $ WatrCont : num 350 435 372 360 204 ...
## $ Substrate : chr "Sphagn1" "Litter" "Interface" "Sphagn1" ...
## $ Shrub : chr "Few" "Few" "Few" "Few" ...
## $ Topo : chr "Hummock" "Hummock" "Hummock" "Hummock" ...
```

We can fit the logistic regression model of the presence of *Galumna* sp. as a function of water content and topography as follows, using the `glm()` function and the `family` argument:

```
logit.reg <- glm(pa ~ WatrCont + Topo,
                 data = mites,
                 family = binomial(link = "logit"))
```

To see the model output, we run:

```
summary(logit.reg)

##
## Call:
## glm(formula = pa ~ WatrCont + Topo, family = binomial(link = "logit"),
## data = mites)
##
## Deviance Residuals:
## Min 1Q Median 3Q Max
## -2.0387 -0.5589 -0.1594 0.4112 2.0252
##
## Coefficients:
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) 4.464402 1.670622 2.672 0.007533 **
## WatrCont -0.015813 0.004535 -3.487 0.000489 ***
## TopoHumock 2.090757 0.735348 2.843 0.004466 **
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 91.246 on 69 degrees of freedom
## Residual deviance: 48.762 on 67 degrees of freedom
## AIC: 54.762
##
## Number of Fisher Scoring iterations: 6
```

Doesn't this structure resembles the one from `summary.lm()`? IT does, right? However you might notice that there are some special differences (e.g. dispersion parameter) we will discuss further in this book!

8.3 Challenge 1

Using the `bacteria` dataset (from the `MASS` package), model the presence of *H. influenzae* as a function of treatment and week of test. Start with a full model and reduce it to the most parsimonious model.

Load the `MASS` package and the `bacteria` dataset:


```
## 'data.frame': 220 obs. of 6 variables:
## $ y : Factor w/ 2 levels "n","y": 2 2 2 2 1 2 2 2 ...
## $ ap : Factor w/ 2 levels "a","p": 2 2 2 2 1 1 1 1 1 ...
## $ h1o: Factor w/ 2 levels "h1","1o": 1 1 1 1 1 1 1 2 2 ...
## $ week: int 0 2 4 11 0 2 6 11 0 2 ...
## $ ID : Factor w/ 50 levels "X01","X02","X03",...: 1 1 1 2 2 2 2 3 3 ...
## $ trt : Factor w/ 3 levels "placebo","drug",...: 1 1 1 1 3 3 3 2 2 ...
```

This dataset was made to test the presence of the bacteria *H. influenzae* in children with otitis media in the Northern Territory of Australia. Dr A. Leach tested the effects of a drug on 50 children with a history of otitis media in the Northern Territory of Australia. The children were randomized to the drug or a placebo. The presence of *H. influenzae* was checked at weeks 0, 2, 4, 6 and 11: 30 of the checks were missing and are not included in this data frame.

► [Click here to see the solution!](#)

8.4 Interpreting the output of a logistic regression

The output of our logistic regression indicates that both water content (`WatrCont`) and topography (`Topo`) are significant:

```
# Extracting model coefficients
summary(logit.reg)$coefficients

##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)  4.46440199 1.670622482  2.672299 0.0075333598
## WatrCont     -0.01581255 0.004535069  -3.486728 0.0004889684
## TopoHummock  2.09075654 0.735348234  2.843220 0.0044660283
```

But how do we interpret the slope coefficients? Remember that we applied a transformation on our expected values (i.e. the probability that $Y = 1$ so we have to use an inverse function to properly interpret the results.

8.4.1 An example using the identity link

If we were to use the identity link function, the interpretation is much easier. Assuming we have a binary outcome y and two covariates x_1 and x_2 and a constant, the probability of a successful outcome ($y = 1$) is given by:

$$Pr(y_i = 1) = p = g^{-1}(\beta_0 + x_{i1}\beta_1 + x_{i2}\beta_2)$$

where $g^{-1}()$ is the **inverse link function**.

For the identity link, the interpretation of the β_1 coefficient is therefore straightforward: For one-unit increase in x_1 , β_1 dictates a constant difference in the outcome.

$$\Delta y_i = (\beta_0 + \beta_1(x_{i1} + 1) + \beta_2x_{i2}) - (\beta_0 + \beta_1x_{i1} + \beta_2x_{i2})$$
$$\Delta y_i = \beta_1$$

8.4.2 Interpreting the coefficients using the logit link

In a linear logistic model with two covariates x_1 and x_2 , we have:

$$\log\left(\frac{p}{1-p}\right) = \beta_0 + \beta_1x_{i1} + \beta_2x_{i2}$$

This corresponds to a log odds ratio! We can use an exponential function to rewrite that model to get the odds ratio:

$$\frac{p}{1-p} = \exp(\beta_0 + \beta_1x_{i1} + \beta_2x_{i2})$$

If we want to convert the odds into probability, given a coefficient α we would use the **inverse logit link function** (also known as the logistic function):

$$Pr(y_i = 1) = \text{logit}^{-1}(\alpha) = \frac{1}{1 + \exp(-\alpha)} = \left(\frac{1}{1 + \exp(-\alpha)}\right) * \left(\frac{\exp(\alpha)}{\exp(\alpha)}\right) = \frac{\exp(\alpha)}{\exp(\alpha) + 1}$$

Now going back to our model, this gives us:

$$Pr(y_i = 1) = \frac{\exp(\beta_0 + \beta_1x_{i1} + \beta_2x_{i2})}{1 + \exp(\beta_0 + \beta_1x_{i1} + \beta_2x_{i2})}$$

Since the inverse link is nonlinear, it is difficult to interpret the coefficient. However, we can look what happens to the differences for a one-unit change to x_1 :

$$\Delta y_i = \frac{\exp(\beta_0 + \beta_1(x_{i1} + 1) + \beta_2x_{i2})}{1 + \exp(\beta_0 + \beta_1(x_{i1} + 1) + \beta_2x_{i2})} - \frac{\exp(\beta_0 + \beta_1x_{i1} + \beta_2x_{i2})}{1 + \exp(\beta_0 + \beta_1x_{i1} + \beta_2x_{i2})}$$
$$\Delta y_i = \exp(\beta_1)$$

As x_1 increases by one unit, the odds increase by a factor of $\exp(\beta_1)$. Note that the odds values here are considered when all other parameters are kept constant.

With this, we can now interpret the results of our model:

```
# model output
logit.reg

##
## Call: glm(formula = pa ~ WatrCont + Topo, family = binomial(link = "logit"),
## data = nites)
##
## Coefficients:
## (Intercept)    WatrCont    TopoHummock
##    4.46440    -0.01581     2.09076
##
## Degrees of Freedom: 69 Total (i.e. Null); 67 Residual
## Null Deviance:      91.25
## Residual Deviance: 48.76    AIC: 54.76
```

For a one-unit increase (or decrease) in our coefficients, we can obtain the odds for the presence of mites.

```
# odds for the presence of mites
exp(logit.reg$coefficient[2:3])

##    WatrCont TopoHummock
## 0.9843118   8.0910340
```

When the odds value is smaller than 1, interpretation is a little bit more complicated. When this is the case, we have to take the inverse value (i.e. 1 divided by the odds) to facilitate interpretation.

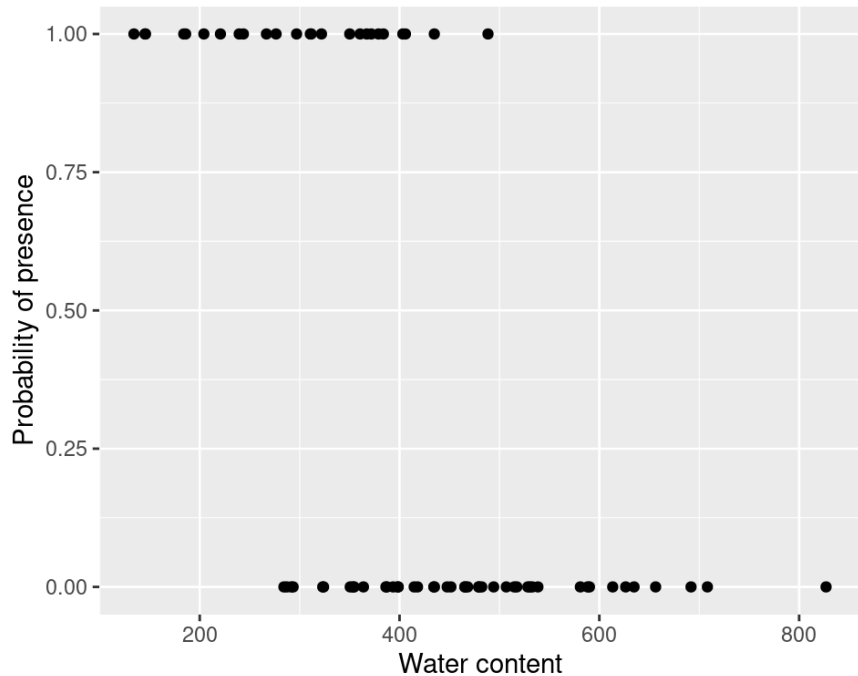
The interpretation is then how **LESS** likely it is to observe the event of interest. For water content, the odds is 0.984. The inverse is:

$$\frac{1}{0.984} = 1.0159$$

This means that a one-unit increase in water content decreases the likelihood of observing *Galumna* sp. by 1.0159. We can also subtract 1 from the odds value to obtain a percentage:

$$(1.0159 - 1) * 100 = 1.59.$$

So there is a 1.59% decrease in probability of observing *Galumna* sp. with a one-unit increase in water content. To convince ourselves that it is an appropriate interpretation, we can plot the presence of *Galumna* sp. as a function of water content. We see that, on average, *Galumna* sp. presence is higher at lower water content than its "absence".



When an estimated parameter is between 0 and 1 on the odds' scale, the relationship between the response variable and the predictors is **negative**. If it is greater than 1, the relationship will be **positive**. If the confidence interval include 1 on the odds' scale, the relationship is *not significant*.

Keep in mind that a value of 1 on the odds' scale indicates that the probability to observe the event Y is equal to the probability of *not* observing the event (i.e. $p = 0.5, 0.5/(1-0.5) = 1$).

Let us try with topography:

```
exp(logit.reg$coefficients[1])

## (Intercept)
##    86.86907
```

The estimated parameter for topography is 2.091 on the odds' scale. Thus, the probability is:

$$1/(1 + 1/\exp(2.091)) = 0.89 \text{ equivalent to } 1/(1 + 1/8.09).$$

Note that the odds for a predictor is calculated when all other variables are constant. Topography has an odd of 8.09. Thus the probability of observing *Galumna* sp. is 8.09 greater when the topography is `hummock` instead of `blanket`.

We can calculate the odds without the function `exp()` :

Let us start with the odds for topography from `logit.reg` :

$$\mu/(1 - \mu) = 8.09$$

We rearrange to isolate μ :

$$\mu = 8.09(1 - \mu) = 8.09 - 8.09\mu$$
$$8.09\mu + \mu = 8.09$$
$$\mu(8.09 + 1) = 8.09$$
$$\mu = 8.09/(8.09 + 1)$$
$$\mu = 1/(1 + (1/8.09)) = 0.89$$

We found the same results !

8.5 Predictive power and goodness-of-fit

An easy and intuitive way to evaluate the predictive power of your model is to compare its deviance to the deviance of a null model. Deviance can be understood as a generalisation of the residual sum of squares when models are estimated by maximum likelihood (i.e. it is how parameters are estimated in GLM). This allows us to compute a pseudo- R^2 statistic, which is analogous to the coefficient of determination R^2 in ordinary least square regression (i.e. the basic method for linear models). The generic formula to compute a pseudo- R^2 is given by:

$$\text{pseudo-}R^2 = \frac{\text{null deviance} - \text{residual deviance}}{\text{null deviance}}$$

where "null deviance" is the deviance of the null model and "residual deviance" is the deviance of the model of interest. The difference is divided by the null deviance so that the result is bound between 0 and 1.

A null model is a model without any predictor. In R, we create a null model this way : `null.model <- glm(response.variable ~ 1, family = binomial) ;`

The unit deviance is a measure of distance between y and μ .

$$d(y, y) = 0$$
$$d(y, \mu) > 0 \quad \forall y \neq \mu$$

The total deviance $D(\mathbf{y}, \hat{\mu})$ of a model with predictions $\hat{\mu}$ of the observation $\mathbf{y}$ is the sum of its unit deviances:

$$D(\mathbf{y}, \hat{\mu}) = \sum_i d(y_i, \hat{\mu}_i)$$

Now, the deviance of a model that has estimates $\hat{\mu} = E[Y|\hat{\theta}_0]$ can be defined by its **likelihood**:

$$D(y, \hat{\mu}) = 2 \Big(\log \big(p(y \mid \hat{\theta}_s) \big) - \log \big(p(y \mid \hat{\theta}_0) \big) \Big)$$

with $\hat{\theta}_0$ denoting the fitted values of the parameters in the reduced model, while $\hat{\theta}_s \hat{\theta}_s$ denotes the fitted parameters for the saturated model.

The **residual deviance** is defined as 2 times the log-likelihood ratio of the full model compared to the reduced model:

$$D(y, \hat{\mu}) = 2 \Big(\log \big(p(\text{saturated model}) \big) - \log \big(p(\text{reduced model}) \big) \Big)$$

And, the **null deviance** is defined 2 times the log-likelihood ratio of the full model compared to the null model (i.e. predictors are set to 1).

$$D(y, \hat{\mu}) = 2 \Big(\log \big(p(\text{saturated model}) \big) - \log \big(p(\text{null model}) \big) \Big)$$

Now we can run this in R. Let us compare the deviance of your model (residual deviance) to the deviance of a null model (null deviance). The **null model** is a model without any explanatory variable and it looks like this:

```
null.model <- glm(response.variable ~ 1, family = binomial)
```

The **saturated (or full) deviance model** is a model with all explanatory variables:

```
full.model <- glm(response.variable ~ ., family = binomial)
```

Residual and null deviances are already stored in the `glm` object :

```
# extract residual and null deviances
objects(logit.reg)
```

```
## [1] "aic"          "boundary"      "call"
## [4] "coefficients" "contrasts"     "control"
## [7] "converged"    "data"          "deviance"
## [10] "df.null"      "df.residual"   "effects"
## [13] "family"       "fitted.values" "formula"
## [16] "iter"         "linear.predictors" "method"
## [19] "model"        "null.deviance" "offset"
## [22] "prior.weights" "qr"            "r"
## [25] "rank"         "residuals"     "terms"
## [28] "weights"      "xlevels"       "y"
```

We can then use these deviance values to calculate the pseudo- R^2 value:

```
# calculate the pseudo-R2
pseudoR2 <- (logit.reg$null.deviance - logit.reg$deviance) / logit.reg$null.deviance
pseudoR2

## [1] 0.4655937
```

Hence, the model explains 46.6% of the variability in the data.

An adjusted McFadden's pseudo- R^2 , which penalizes for the number of predictors, can be calculated as below:

$$R^2_{adj} = 1 - \frac{\log L(M) - K}{\log L(M_{null})}$$

where **K** corresponds to the additional number of predictors in relation to the null model.

The goodness-of-fit of logistic regression models can be expressed by variants of *pseudo* – R^2 statistics, such as Maddala (1983) or Cragg and Uhler (1970) measures.

When talking about logistic regressions, low R^2 values are common.

The R function `DescTools::PseudoR2()` makes it possible to calculate many types of *pseudo* – R^2 . By specifying `which = all`, calculate all of them at once.

```
# Calculate many pseudo-R2!
logit.reg <- glm(pa ~ MatrCont + Topo,
  data = mites,
  family = binomial(link = "logit"))
DescTools::PseudoR2(logit.reg, which = "all")
```

```
##      McFadden      McFaddenAdj      CoxSnell      Nagelkerke      AldrichWelson
## 0.4655937      0.3998373      0.4549662      0.6245898      0.3776866
## VeallZimmermann      Efron      McKelveyZavoina      Tjur      AIC
## 0.6674318      0.5824181      0.7864893      0.5114661      54.7623962
##      BIC      logLik      logLik0      G2
## 61.5878819      -24.3811981      -45.6229593      42.4835224
```

8.5.1 Challenge 2

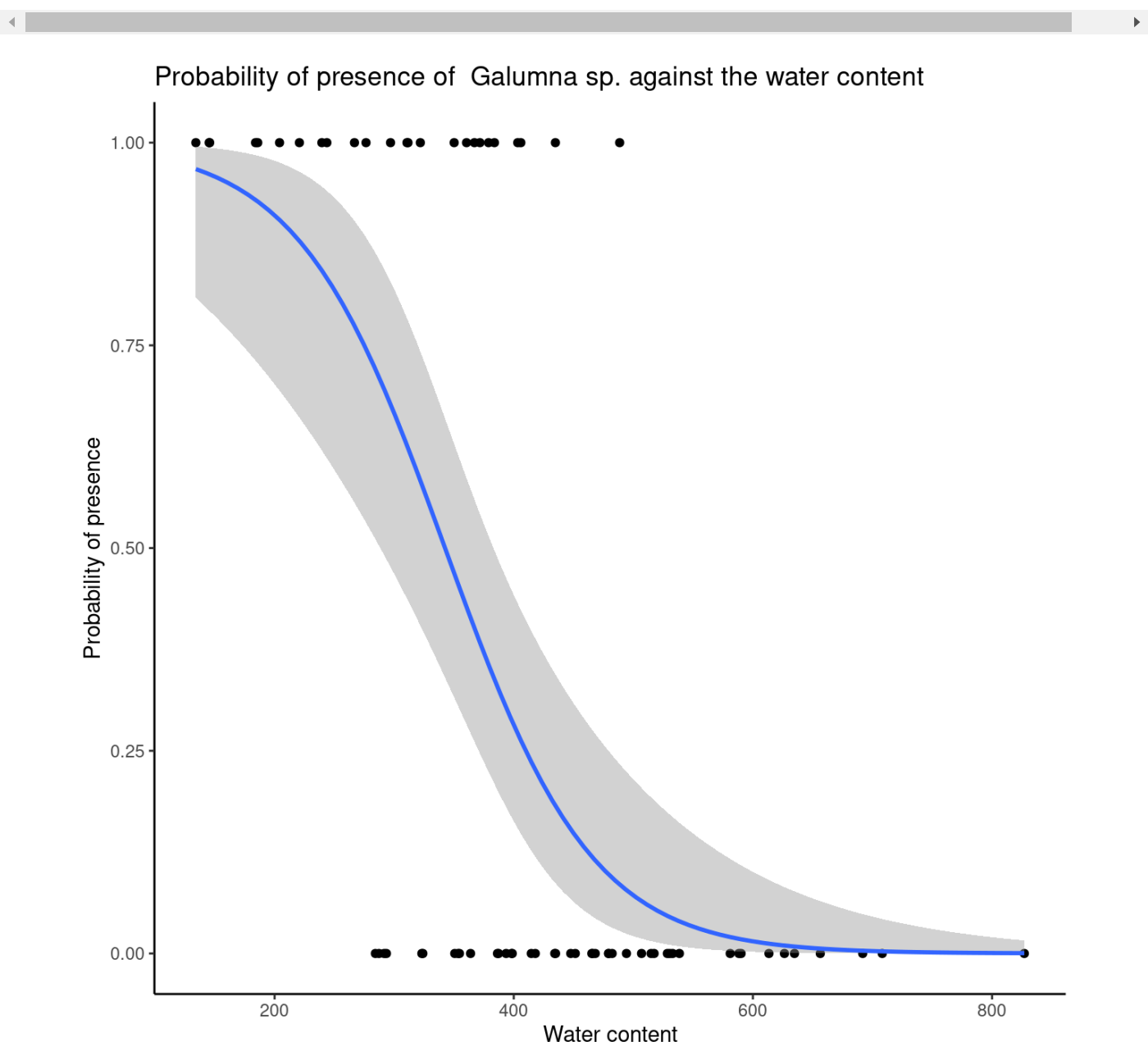
Assess goodness-of-fit and predictive power of the `model.bact2` model. How can you improve the predictive power of this model?

► [Click here to see the solution!](#)

8.6 Visual representation of results

Once we have validated our model, we would likely want to visually represent our results. Here is an example with `ggplot2`. See [workshop 3](#) for a reminder for this package.

```
ggplot(mites, aes(x = MatrCont, y = pa)) + geom_point() +
  stat_smooth(method = "glm", method.args = list(family=binomial), se = TRUE) + xlab("Water content")
ylab("Probability of presence") +
  ggtitle("Probability of presence of Galumna sp. against the water content")+theme_classic()
```



All the content of the workshop series is under a [Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License](#).